

Model Card & Deployment Guide — Photodamage DermLIP

MTL v4

Ship this document with the packaged model. It specifies the architecture, the exact input/output contract, preprocessing, a runnable usage example, and licensing — everything a collaborating site needs to deploy the model correctly.

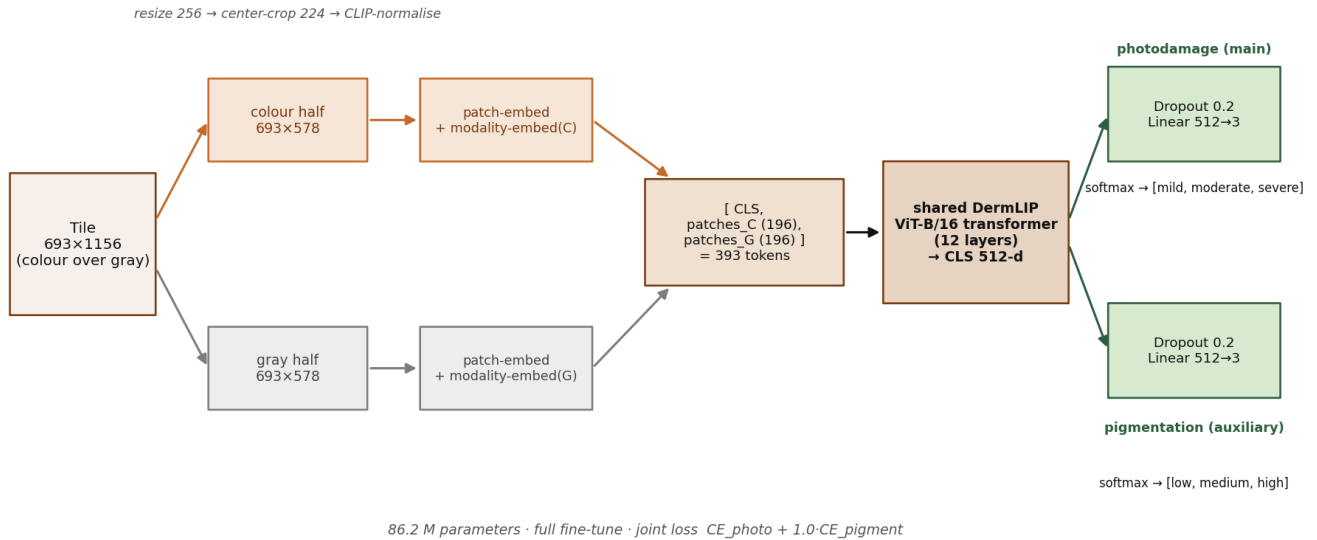
Model name	photodamage-dermlip-mtl-v4
Version / date	v4, 2026-07-06
Checkpoint	dermlip_best_sw3.0_seed42/best.pt (344 MB, 86.2 M params)
Task	Per-tile 3-class photodamage severity (main) + 3-class pigmentation (auxiliary)
Internal test macro-AUC	0.930 (seed 42; 5-seed mean pending)
Framework	PyTorch + open_clip
Backbone licence	DermLIP / Derm1M is CC BY-NC 4.0 — non-commercial research use only (see §8)

1 Intended use

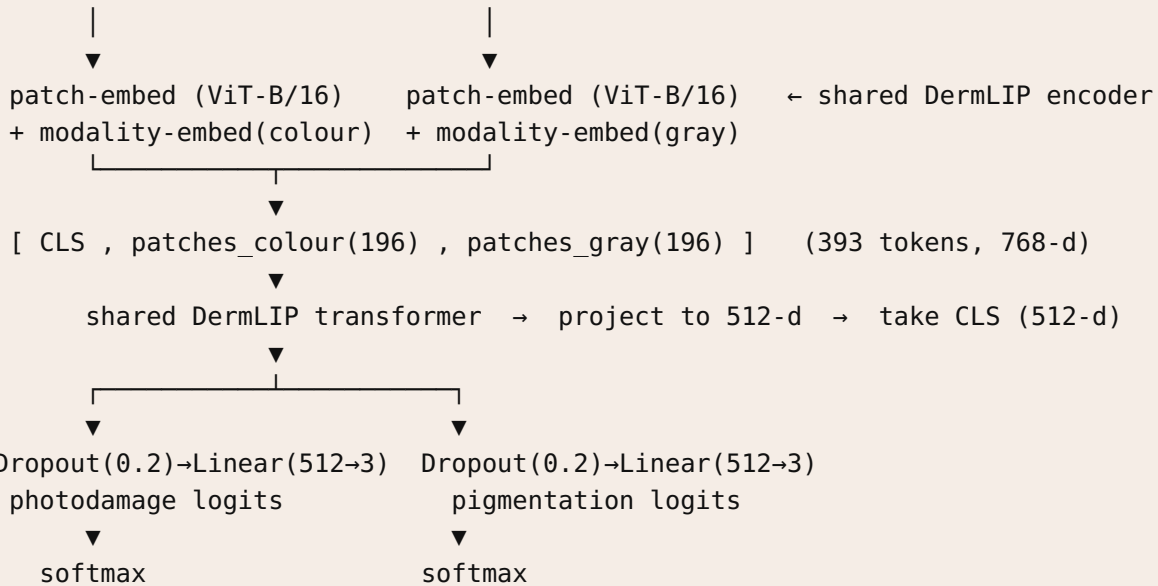
- **Input:** dermatological skin tiles (~10 cm² of cutaneous surface) derived from a Canfield **Vectra WB360** 3D total-body-photography session, following the tiling pipeline in §3.
 - **Output:** per-tile photodamage and pigmentation severity distributions, aggregated to a **patient-level severity score** for risk stratification (§6).
 - **Not intended for:** standalone diagnosis, non-WB360 imagery without re-validation, or any commercial deployment (backbone licence).
-

2 Architecture

DermLIP dual-stream multi-task architecture (photodamage + pigmentation)



Two-stream input (colour tile half, grayscale tile half)



- **Backbone:** DermLIP ViT-B/16 (CLIP-style image encoder, `open_clip`), internal dim 768, projects to 512.
- **Fusion** (`modality-embed`): colour and gray tile halves are patch-embedded, patches receive learnable per-modality embeddings, and a single CLS token attends over both patch sets in one shared transformer pass. Output = the 512-d CLS feature.
- **Heads:** two independent `Dropout(0.2) → Linear(512→3)` heads (photodamage main + pigmentation auxiliary), trained jointly.
- **Parameters:** 86.2 M (full fine-tune).

3 Input contract (READ CAREFULLY — site reproducibility depends on it)

The model consumes **one tile at a time**, but the tile must be produced by the exact upstream chain:

1. **WB360 capture** → ~92 raw cross-polarised 2D camera views (`a##B.CR2`).
2. **CR2** → **JPG** via Canfield `convertCR2.exe` , producing a colour JPG **and** a grayscale companion (`_gray`).
(Windows-only tool — see the acquisition pipeline; do not substitute a different RAW converter without re-validation.)
3. **Tiling** → 9×5 grid of **693 × 693 px** tiles (≈10 cm² each); reject any tile with < 33 % skin surface (skin mask).
Colour and gray tiles are stacked vertically into one **693 × 1156 px** file: **colour = top half, grayscale = bottom half**.
4. **Head-region tiles excluded** (de-identification), matching the source study.

Per-tile preprocessing (must match exactly)

```
tile (693 × 1156 RGB)
colour_half = tile.crop((0, 0, 693, 578))      # top
gray_half   = tile.crop((0, 578, 693, 1156))    # bottom
for each half:
    resize to (256, 256) → center-crop (224, 224) → ToTensor
    normalize with CLIP mean/std:
        mean = (0.48145466, 0.4578275, 0.40821073)
        std  = (0.26862954, 0.26130258, 0.27577711)
    model input = (colour_tensor[3,224,224], gray_tensor[3,224,224])
```

Filename convention (carries metadata used for aggregation): `<patient_id>_<camera>B_<grid_cell>.jpg` ,
e.g. `7775-542_a10B_b3.jpg` .

4 Output contract

`model(colour, gray)` returns two logit tensors (`logits_photodamage[3]` , `logits_pigmentation[3]`).
Apply softmax to each:

Head	Index 0	Index 1	Index 2
Photodamage (main)	mild	moderate	severe
Pigmentation (auxiliary)	low	medium	high

Report the **full probability vector**, not just the argmax — it is what downstream aggregation uses.

5 Usage (minimal, runnable)

Package requirements: `torch>=2.2` , `open_clip_torch` , `Pillow` , `numpy` , and the shipped `models.py` .

```

import torch
from PIL import Image
from torchvision.transforms import functional as TF
from models import build_model          # shipped with the package

CLIP_MEAN = (0.48145466, 0.4578275, 0.40821073)
CLIP_STD  = (0.26862954, 0.26130258, 0.27577711)

def preprocess_half(img):
    img = TF.resize(img, (256, 256))
    img = TF.center_crop(img, 224)
    return TF.normalize(TF.to_tensor(img), CLIP_MEAN, CLIP_STD)

# ---- build architecture and load fine-tuned weights ----
# NOTE: build_model calls open_clip to instantiate the DermLIP ViT-B/16 scaffold;
# our checkpoint then overrides ALL weights. First run needs HuggingFace access to
# `redlessone/DermLIP_ViT-B-16` (see §8 offline note).
model = build_model("dermlip", mode="full", dual_stream=True, fusion="modality-
embed",
                    head_dropout=0.2, n_photodamage=3, n_pigmentation=3)
ckpt = torch.load("best.pt", map_location="cpu")
model.load_state_dict(ckpt["model"])
model.eval().cuda()

# ---- score one tile ----
tile = Image.open("7775-542_a10B_b3.jpg").convert("RGB")    # 693 x 1156
w, h = tile.size
colour = preprocess_half(tile.crop((0, 0, w, h // 2))).unsqueeze(0).cuda()
gray   = preprocess_half(tile.crop((0, h // 2, w, h))).unsqueeze(0).cuda()

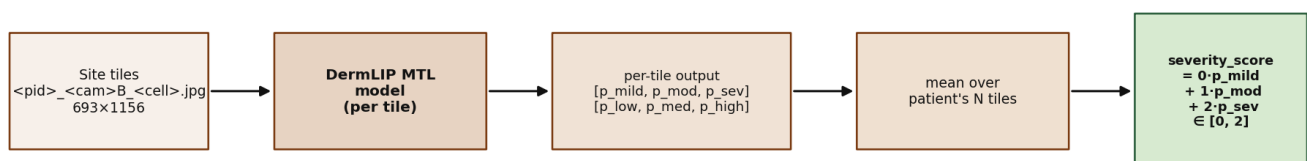
with torch.no_grad():
    logit_photo, logit_pig = model(colour, gray)
    p_photo = logit_photo.softmax(-1)[0].cpu()    # [p_mild, p_moderate, p_severe]
    p_pig   = logit_pig.softmax(-1)[0].cpu()      # [p_low, p_medium, p_high]
    print("photodamage:", p_photo.tolist(), "pigmentation:", p_pig.tolist())

```

A batched reference implementation over a whole cohort is in `pipelines/04_infer_sundamage.py`.

6 Patient-level risk aggregation

From tiles to patient-level risk



`per_tile_predictions.csv` → `per_patient_severity.csv` (`run_inference.py`)

```
For each patient with N accepted tiles:
    mean_p_c      = mean over tiles of p_c          for c in {mild, moderate,
severe}
    severity_score = 0*mean_p_mild + 1*mean_p_moderate + 2*mean_p_severe    # in [0,
2]
    modal_class    = majority argmax class across tiles
```

`severity_score` (continuous, [0,2]) is the recommended single risk value per patient. Reference: `pipelines/05_aggregate_external.py`.

7 Package contents (recommended bundle)

```
photodamage-dermlip-mtl-v4/
├── best.pt                # fine-tuned checkpoint (344 MB)
├── config.json            # exact training config for provenance
├── models.py              # architecture (build_model + DualStreamMTL)
├── MODEL_CARD.md          # this document
├── example_infer.py        # the $5 snippet, runnable
└── requirements.txt        # torch>=2.2, open_clip_torch, Pillow, numpy
```

8 Licensing, offline deployment, and limitations

- **Licence — IMPORTANT:** the DermLIP / Derm1M backbone is **CC BY-NC 4.0 (non-commercial)**. This model may be used for **research and internal evaluation only**, not commercial deployment. Any distribution must retain this notice. For a commercially deployable artefact, use the ResNet50 variant instead (ImageNet-licensed) or negotiate a commercial DermLIP licence.
- **Offline / air-gapped sites:** `build_model` downloads the DermLIP ViT-B/16 scaffold from HuggingFace on first construction (our checkpoint then overrides every weight). Pre-cache `redlessone/DermLIP_ViT-B-16` (set `HF_HOME`) on a connected machine and copy the cache to the target site.
- **Domain:** validated on Vectra WB360 cross-polarised tiles. Other cameras, lighting, or tiling schemes require re-validation.
- **Statistical caveat:** internal test AUC 0.930 is a **single seed** on a small student-patient eval pool (10 patients); treat as provisional until the 5-fold mean $\pm$ SD is reported.
- **Not a diagnostic device:** outputs are a research risk signal, not a clinical diagnosis.

Model developed by the s2-sun_damage team, reproducing Kahler et al. (Br J Dermatol 2026;194:737–746) and benchmarking the DermLIP/Derm1M foundation model (Yan et al.). Full results in `V4_RESULTS_REPORT.md`.